

The Agent Ontology: A Static, Signed, Fabrication-Safe Crosswalk From Agent Actions to Governance Controls

Cyber Warrior Network

Draft v2 - preprint, targeting arXiv cs.CR (cross-list cs.AI / cs.SE). Not peer reviewed. v2 adds measured figures, a formal claims/non-claims and invariants section, a related-work comparison, additional measured experiments (determinism, classification accuracy, a non-agentic false-positive baseline, a tamper matrix), post-quantum receipt signing in the open-source tool, and the graph-grounded resolver that v1 listed as future work.

Abstract

Autonomous agents execute side-effecting actions faster than human review, yet most deployed agents have no machine-readable record of which governance control answers for each action. We present OpenAgentOntology (OAO), an open-source tool and standard that reads an agent's source (AST and text, never executing it) and emits a typed capability graph; a confidence-tagged crosswalk from each side-effecting action to NIST SP 800-53r5, the EU AI Act, the OWASP LLM Top 10 (2025), the NIST AI RMF, and MITRE ATT&CK; a four-tier Trust Profile; and a cert-only receipt that verifies offline and fails on a single altered byte. The receipt is hybrid-signed: Ed25519 always, plus post-quantum ML-DSA-65 (FIPS 204) and SLH-DSA (FIPS 205) legs when an optional extra is installed. ASSERTED controls require a source-declared canonical reason from a fixed ten-reason table; everything auto-detected is INFERRED or AMBIGUOUS; no code path constructs a framework id, a property we establish structurally and not merely by test. We scanned five headline agents and report measured figures throughout. The two most-starred autonomous coding agents on GitHub, open-interpreter (~58k stars) and gpt-engineer (~54k stars), carry zero declared governance bindings: every control surfaced for their 21 and 6 side-effecting actions had to be inferred from a verb name (11 INFERRED + 9 AMBIGUOUS and 3 INFERRED + 2 AMBIGUOUS respectively), and the most dangerous action in each (`exec`, arbitrary code execution, MITRE ATT&CK T1059; `post_data`) is ungoverned even by that inference. Both tier UNGOVERNED, an assignment unchanged under a measured coverage-weight sweep from 0.35 to 0.55. We add measured controls: the pipeline is deterministic (8/8 re-scans reproduce the evidence hash), it predicts the correct tier on 7/7 in-repo agents with known ground truth and the correct governance state on a 10/10 hand-labeled action set, it raises no spurious action surface on a 12-file non-agentic baseline (false-positive rate 0.0), and both committed receipts verify and both fail under a one-field tamper. An autonomous remediation agent lifts a fresh open-interpreter clone from UNGOVERNED to a projected DEVELOPING tier, declining to auto-assert nine ambiguous `run_*` actions. A deterministic lattice boundary (abstract-interpretation over the canonical-reason set) turns each graph-grounded resolution into a verified-or-abstain gate verdict -- PROCEED, PROCEED-FLAGGED, or ESCALATE to a human when governance is not grounded enough to act on; on the open-interpreter scan it escalates 15 of 21 actions. We are explicit about what we did not benchmark: extraction precision/recall on the two real-world agents (no ground-truth labels), calibration of the graph-grounded layer, multi-annotator agreement, and large-corpus prevalence remain the open benchmark program. OAO turns the governance gap into a tier and a signed artifact; it is the static accountability layer that composes with, and does not replace, a runtime policy engine.

1. Introduction

An autonomous agent is, operationally, a new employee you did not interview, that works at machine speed, and that you cannot fire. It can move money, export regulated records, ship code to production, and shut systems down. For a human with that authority, an organization maintains an explicit record of which control answers for each sensitive action: separation of duties on the wire transfer, an approval gate on the production change, a named owner on the high-blast-radius operation. For the agent doing the same work, that record usually does not exist in any form a machine can read - it lives, if anywhere, in a slide, a spreadsheet that is already stale, or someone's memory.

This is the governance gap. The agent stack was built in three conceptual layers, and the industry shipped two of them. The first layer is the agent frameworks (LangChain, CrewAI, AutoGen, MCP) that optimize capability - how to build agents that can do more. The second is observability (tracing, evals, OpenTelemetry-style instrumentation) that optimizes hindsight - what the agent did on a given run. The missing third layer is accountability: for each thing the agent can do, *which control answers for it, before it runs*. Capability does not measure that. Observability measures it only after the fact, one execution at a time, and only for the path that happened to execute. Neither layer can tell you, statically, that an agent which can `exec` arbitrary code has no gate on that capability at all.

We call the missing layer the Agent Ontology: a typed, signed map of every action an agent can take and the governance control that answers for each one. The reframe that makes it tractable is that a single guardrail is not "a Rego rule" - a wire-transfer dual-control check is, simultaneously, NIST 800-53 AC-5 (Separation of Duties), EU AI Act Article 14 (Human oversight), and OWASP LLM06 (Excessive Agency). Auditors, regulators, and boards each speak a different one of those vocabularies. The Agent Ontology maps an action once, to a shared internal vocabulary, and emits the right control identifier for whichever framework the reader cares about - so the same control is not re-translated five ways by hand.

Accountability is not enforcement, and that separation is deliberate. OAO is a static instrument: it tells you which actions have a control declared and which have none. It does not gate an action at runtime, and it should not - the thing that *runs* an agent cannot credibly grade its own governance, just as a trading desk's controls are not attested by the desk. Enforcement belongs in a runtime policy engine (an OPA-style gate that returns allow/deny before the action executes); OAO is the pre-flight inspection that finds the doors with no lock. The two compose directly: OAO ingests a policy engine's deny reasons as input, and an action sitting behind a gate with a canonical deny reason scores ASSERTED (Section 3.2). In deployment terms, OAO is the open, independent map; a runtime trust gate is the lock. This paper is about the map.

The framing we adopt throughout is the insider threat onboarded on purpose. An agent with `exec` and no gate is not a malicious insider, but it has an insider's access and an insider's blast radius, and it was granted that access deliberately, with no compensating control recorded. The point of OAO is not to assert that any scanned agent is malicious or even badly built - the agents we scan are excellent engineering. The point is that the mapping from their actions to the controls that should answer for them is absent from their own source, which makes the gap invisible until something measures it.

Measured in the wild, the gap is stark. The two most-starred autonomous coding agents on GitHub declare no governance bindings at all: every control OAO surfaces for them is an inference from a verb name, and the most dangerous action in each answers to nothing - not even a guess (Section 7). We state up front, and explain in Section 3.2, that a zero-asserted result is structurally guaranteed for any agent that does not declare canonical reasons; the finding is precisely that this declaration layer does not exist in the wild.

Contributions

1. **A small standard for the Agent Ontology** (Section 3): 13 node types and 13 edge types drawn as the static-observable subset of a Universal Agentic Ontology, with provenance on every node, edge, and mapping; a formal model and ten invariants (Section 4).
2. **A three-layer, fabrication-safe crosswalk** from agent actions to five real governance frameworks: an ASSERTED table of ten canonical deny reasons, an INFERRED verb-heuristic downgrade layer, and an explicit UNGOVERNED outcome - structured so no code path can construct a control id (Sections 3, 4).
3. **A MITRE ATT&CK advisory-enrichment layer** that attaches the adversary technique a SOC already hunts to each canonical reason, by construction never counted toward the score (Section 3).
4. **A cert-only, hybrid post-quantum receipt** (Ed25519 + ML-DSA-65 + SLH-DSA) with byte-for-byte offline verification and a demonstrated tamper matrix, so a passing grade cannot be forged (Sections 3, 7).
5. **A measured evaluation** of five agents, with figures: a governance-state breakdown, a subscore breakdown, a weight-sensitivity sweep, a remediation before/after, a non-agentic false-positive baseline, plus determinism, tier-prediction accuracy, action-state accuracy, and a tamper matrix - and an explicit statement of the benchmark program still open (Sections 7, 8).
6. **A graph-grounded semantic resolver** (the layer v1 listed as future work), which resolves an action through a live governance knowledge graph to candidate canonical reasons and live adversary signals, always at a GRAPH_INFERRED confidence that can never become ASSERTED (Section 9).

Everything in this paper is reproducible from the committed artifacts and the pinned upstream commits; every number is either directly measured by running the tool (`paper/experiments.py`) or transcribed from a committed signed scan, and projected quantities are labeled.

2. Background and related work

OAo sits beside several established bodies of work; it does not replace any of them. We cite standards and frameworks by their own published identifiers and do not attribute claims to invented secondary literature.

AI TRiSM (Trust, Risk and Security Management). The industry framing of AI trust as a lifecycle concern - governance, observability, security, and model operations - motivates a static accountability layer that complements runtime controls. OAo occupies the governance-mapping slice: it does not enforce, observe at runtime, or operate models; it maps actions to controls before runtime.

NIST SP 800-53r5 and the NIST AI RMF. OAo's asserted control identifiers are drawn from NIST SP 800-53 Revision 5 - among them AC-3 (Access Enforcement), AC-4 (Information Flow Enforcement), AC-5 (Separation of Duties), AC-6 (Least Privilege), CM-3 (Configuration Change Control), CM-4 (Impact Analyses), CM-5 (Access Restrictions for Change), RA-2 (Security Categorization), and SC-7 (Boundary Protection). The NIST AI RMF functions (GOVERN, MAP, MEASURE, MANAGE) appear as advisory mappings only, because a risk-management function is a posture, not an exact technical control.

EU AI Act (Regulation (EU) 2024/1689). OAo maps human-oversight obligations to Article 14 and data-governance obligations to Article 10. The crosswalk's asserted mappings in this version use Articles 10 and 14; the product layer also references Article 50 (transparency).

OWASP LLM Top 10 (2025). OAo uses LLMo2 (Sensitive Information Disclosure) for regulated-egress actions and LLMo6 (Excessive Agency) for unbounded-authority and weak-heuristic actions. LLMo6 is the single stub a weak/overloaded verb can resolve to, and never a specific 800-53 identifier.

MITRE ATT&CK. Each canonical reason carries the adversary technique it mitigates - e.g. T1059 (Command and Scripting Interpreter), T1048 (Exfiltration Over Alternative Protocol), T1485 (Data Destruction), T1543 (Create or Modify System Process), T1078 (Valid Accounts) - so a SOC analyst reads the finding in the language they already hunt in. These are advisory and never inflate the score.

2.1 Where OAO sits among adjacent artifacts

The closest neighbors are not other "agent governance" products but the standardized artifacts of security and supply-chain engineering. OAO is to agent actions what each of these is to its own domain; it is not a replacement for any of them, and it deliberately borrows their design discipline (a fixed schema, signed evidence, offline verification).

System	What it answers	What it does <i>not</i> answer	Where OAO fits
OPA / Rego	"Is <i>this request</i> allowed, right now?"	which actions have <i>no</i> policy at all	OAO ingests Rego deny reasons; an action behind an OPA gate scores ASSERTED. OPA is the lock; OAO finds the doors with no lock.
OSCAL	machine-readable control <i>catalogs</i> and assessment plans	the agent's action surface	OAO emits control ids OSCAL catalogs define; alignment target, not overlap.
SARIF	a transport format for <i>static-analysis findings</i>	governance mapping or a trust grade	OAO emits SARIF so its ungoverned/ambiguous findings drop into existing code-scanning pipelines.
CycloneDX / SPDX (SBOM)	what <i>components</i> an artifact contains	what <i>actions</i> an agent can take and who governs them	OAO is the "bill of governance" analogue for the agent action surface.
Sigstore / SLSA	provenance and signing of <i>build artifacts</i>	per-action governance	OAO's receipt borrows the signed-evidence/offline-verify discipline and applies it to a governance ontology.
OpenTelemetry / LangSmith (tracing)	what the agent <i>did</i> on a run	what it <i>can</i> do and which control answers, before it runs	tracing is the flight recorder; OAO is the pre-flight inspection. Naming-convention inspiration, not equivalence.
MCP tool registries	what tools an agent <i>exposes</i>	the governance binding for each tool action	OAO ingests MCP manifests as one input and maps each tool action to controls.
Agent-framework guardrails	runtime input/output checks	a static, independent, cross-framework grade	a control plane that runs the agent cannot independently grade its own governance; OAO is the independent grader.

The novel position OAO occupies is the static, fabrication-safe, signed crosswalk itself: a single artifact that is typed (so it is a graph, not a list), multi-framework (so each reader gets their language), honesty-tiered (so a guess is never dressed as an assertion), and tamper-evident (so a grade cannot be forged). We are not aware of an existing open tool that produces exactly this artifact for arbitrary agent source.

3. Method

OAo is a deterministic pipeline: a path goes in, a governed result comes out. The stages are ingest -> generate -> validate -> score -> receipt -> badge. No stage executes the ingested code, and the core pipeline makes no network call.

3.1 The ontology vocabulary

The schema is the static-observable subset of a Universal Agentic Ontology. It defines **13 node types** - Actor, Agent, Capability, Tool, Task, Workflow, Decision, Policy, Gate, Evidence, Outcome, Resource, Domain - and **13 edge types** - OWNS, DELEGATES_TO, HAS_CAPABILITY, USES, EXECUTES, PART_OF, PRODUCES, MAKES, GOVERNED_BY, SUPPORTED_BY, ENFORCES, GATED_BY, OPERATES_ON. A node is {id, type, name, props, provenance} ; an edge is {src, rel, dst, provenance} . Every node, edge, and mapping carries a provenance value - EXTRACTED (explicit in the source), INFERRED (a strong single-domain verb matched a heuristic), or AMBIGUOUS (only a weak/overloaded verb matched). The assembled OntologyDoc serializes deterministically (nodes sorted by id, edges by (src, rel, dst) , action maps by subject id, frameworks sorted), which is what makes the receipt hash reproducible across runs and across languages.

Ingest reads repos (extracting tool decorators such as @tool , @function_tool , @mcp.tool , and verb-named functions such as exec , run_* , delete_* , send_* , deploy_*), MCP manifests, LangChain/CrewAI specs, OpenAPI documents, Rego policy, and workflow YAML. Generate assembles the typed graph and drops any dangling edge (an edge whose endpoint is not a node); if one survived into validation it would fail closed.

3.2 The three-layer crosswalk

Each Capability/Decision/Gate or side-effecting action is resolved by map_action(label, source_reason) in a fixed order.

Layer 1 - ASSERTED table (exact). If the source named a reason (a Rego deny key, a guardrail name, an eval expected reason) and that reason is one of the ten canonical reasons, OAo returns that reason's control mappings verbatim, at provenance EXTRACTED, with the table's asserted / advisory confidence untouched, and matched_via = asserted_table . The ten canonical reasons and a representative asserted control each:

canonical reason	representative asserted controls
dual_control_required	NIST 800-53 AC-5; EU AI Act Art 14; OWASP LLM06
over_threshold	NIST 800-53 AC-3, AC-6; EU AI Act Art 14; OWASP LLM06
beneficiary_unverified	NIST 800-53 AC-3; EU AI Act Art 10; OWASP LLM06
human_review_required	EU AI Act Art 14; OWASP LLM06
classification_above_ceiling	NIST 800-53 RA-2, AC-4; EU AI Act Art 10; OWASP LLM02
out_of_scope_domain	NIST 800-53 AC-6, AC-3; OWASP LLM06
regulated_egress_blocked	NIST 800-53 AC-4, SC-7; EU AI Act Art 10; OWASP LLM02
change_freeze_active	NIST 800-53 CM-3, CM-5; OWASP LLM06
approval_required	NIST 800-53 CM-3; EU AI Act Art 14; OWASP LLM06
high_blast_needs_named_approver	NIST 800-53 CM-4, AC-5; EU AI Act Art 14

Layer 2 - heuristic (inferred or ambiguous). With no source-named reason, the action label is run against an ordered list of verb regexes. A strong, single-domain verb (`pay`, `export`, `delete`, `deploy`, `migrate`, `approve`, `terminate`, `escalate`, ...) selects the matching canonical reason but **downgrades** its asserted controls to `confidence = inferred`, `provenance = INFERRED`, with the basis rewritten to name the inferring verb. A weak or overloaded verb (`process`, `update`, `run`, `execute`, `modify`, `write`, `store`, `submit`, `commit`, ...) yields at most a single OWASP LLMo6 stub at `confidence = ambiguous`, `provenance = AMBIGUOUS` - never a specific 800-53 id. The regex boundary is deliberately non-`\b` so a verb matches at the head of a snake_case label (`export_records`, `wire_transfer`).

Layer 3 - UNGOVERNED. A label that matches neither layer returns empty mappings, `matched_via = none`, and the validator emits a non-blocking `W_UNGOVERNED` warning. This is the honest outcome for `exec`-like raw verbs that the strong heuristic deliberately refuses to claim a clean 1:1 control for.

The fabrication invariants. Two properties hold by construction: (1) `map_action` never constructs a framework-id string - it only re-emits identifiers that already live in the asserted table, so there is no path to invent `AC-99`; and (2) `asserted` / `advisory` confidence is reserved for Layer-1 source-named matches, so anything auto-detected is `inferred` or `ambiguous`. The badge shows asserted controls only. These invariants are structural, not merely tested: the set of emittable asserted identifiers is exactly the range of a literal table that the heuristic downgrade path also copies from, independent of input. The test suite corroborates this mechanically (Section 6), but the guarantee does not rest on the tests.

What a zero-asserted result can and cannot mean. Because asserted confidence requires a source-declared canonical reason, an agent that was not written against this (or any) declaration vocabulary is structurally guaranteed to score zero asserted controls. A zero-asserted result for a third-party agent is therefore not, by itself, a discovery about that agent's quality; it is the measurement that the declaration layer - any machine-readable binding from actions to controls - does not exist in that agent's source. The findings that are *not* structurally guaranteed, and that Section 7 leans on, are (a) which actions remain ungoverned even under the inference layer, and (b) the size and shape of the inferred-only surface that someone must now confirm by hand.

3.3 MITRE ATT&CK advisory enrichment

Each canonical reason additionally carries one MITRE ATT&CK technique at `advisory` confidence (e.g. `regulated_egress_blocked` -> T1048; `change_freeze_active` -> T1485; `approval_required` -> T1543; `dual_control_required` -> T1078; `human_review_required` -> T1059). Advisory mappings are informative for a SOC reader but are never counted toward coverage, rigor, or breadth, so attaching the adversary technique cannot inflate the trust score.

3.4 Validation (fail-closed)

`validate(doc)` returns `(ok, findings)`; `ok` is false if any error-level finding exists. Error codes void the artifact: `E_EMPTY` (zero nodes), `E_NODE_TYPE` (unknown node type), `E_EDGE_REL` (unknown edge rel), `E_DANGLING` (edge endpoint not a node), `E_FRAMEWORK` (a mapping framework outside the allowed set), `E_ASSERTED` (an asserted mapping missing `fw` / `id` / `name` / `basis`), `E_FAKE_ID` (an asserted `(fw, id)` pair not in the asserted table), `E_MATCHED_VIA` (a `matched_via = asserted_table` carrying zero mappings - a claim without evidence), and `E_NONASCII` (non-ASCII anywhere that flows into signed evidence). `E_FAKE_ID` and `E_MATCHED_VIA` are the backstops against a forged upgrade. Warn codes (`W_UNGOVERNED`, `W_AMBIGUOUS`) do not block but feed the Trust Profile.

3.5 The receipt: hybrid post-quantum, cert-only, offline-verifiable

The receipt is a tamper-evident record of one scan. The evidence object is the deterministic `OntologyDoc.to_dict()` plus a compact summary header. Canonical JSON is `json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=True)`. The receipt computes `evidence_hash = sha256(canon(evidence))`, builds a small body that commits to that hash (atom_id, type, decision, evidence_hash, signed_at), and signs `canon(body)`.

The signature is **hybrid**. An Ed25519 leg is always present (fast, small, RFC 8032). When the optional cryptographic extra is installed, two **post-quantum** legs are added over the identical body bytes: **ML-DSA-65** (NIST FIPS 204, lattice, NIST Level 3) and **SLH-DSA-SHA2-128s** (NIST FIPS 205, hash-based - a diversity leg that survives a lattice break). The motivation is harvest-now-decrypt-later: a receipt is long-lived evidence, and an adversary who stores it today should not be able to forge it the day the curve falls. The legs are strictly additive - they sign the same `canon(body)`, are written to new fields, and never alter the evidence hash or the Ed25519 body - so an older Ed25519-only receipt still verifies unchanged, and a verifier with no post-quantum backend simply checks the legs it can. Degradation is tiered and honest: with liboqs the host signs `Ed25519+ML-DSA-65+SLH-DSA`; with a pure-Python ML-DSA backend it signs `Ed25519+ML-DSA-65`; with neither it signs Ed25519 only and says so. If even `cryptography` is absent, the receipt is emitted unsigned and explicitly flagged - the tool never fakes a signature.

Verification needs no network and no database: recompute the evidence hash and compare, then verify each present signature leg over the rebuilt body with the embedded public key. Because everything in evidence is ASCII and the serialization is sorted, a browser or any compatible tool reproduces the Python hash byte-for-byte; a JavaScript canonicalizer in the test suite reproduces it independently. The tamper property is the point: edit any field of the evidence - for instance, flip a `matched_via` from `none` to `asserted_table` to fake a better grade for `exec` - and the recomputed `evidence_hash` no longer matches, so verification returns `ok: False` with reason "evidence_hash mismatch." A passing grade cannot be forged without breaking the receipt. We demonstrate this on real committed receipts in Section 7.4.

3.6 The Trust Profile: four tiers, four weighted subscores

The score is a weighted sum of four subscores, each in 0-100:

- **coverage (weight 0.45)** - of the governed actions, the fraction with at least one ASSERTED control. This dominates because an ungoverned high-risk action is the entire failure mode.
- **rigor (weight 0.25)** - of the actions that mapped to anything, the fraction that are asserted rather than heuristic-only. This penalizes guesswork so weak verb matches cannot inflate a grade.
- **breadth (weight 0.15)** - distinct asserted frameworks present, capped at a ceiling of 4 so breadth cannot paper over thin coverage.
- **structure (weight 0.15)** - is the graph typed and connected (edges present, every endpoint resolves, more than one node)?

The four tier bands are **SOVEREIGN** ≥ 90 , **HARDENED** ≥ 75 , **DEVELOPING** ≥ 50 , **UNGOVERNED** < 50 . An ontology with zero governed actions cannot earn a governed tier: coverage and rigor are defined as 0, so it floors at UNGOVERNED. The score and tier live inside the signed evidence, so the badge cannot be inflated after the fact without breaking the signature. The weights and floors are design choices, not values derived from an external ground truth; for that reason this paper treats the **tier as the primary reading** and the integer score as a secondary index, and Section 7.3 reports a measured sensitivity analysis.

The newer v0.2 standard additionally emits a **split risk profile** with four separate subscores (declaration coverage, enforcement evidence, static-extraction confidence, and severity-weighted risk exposure) and a SARIF 2.1.0 finding stream; these are additive and do not change the four-tier Trust Profile used throughout this paper.

4. Formal model and invariants

We give a compact model so the guarantees can be stated precisely rather than only in prose.

Let A be the set of observable side-effecting actions extracted from a target, R the fixed set of ten canonical reasons, C the fixed set of asserted framework control identifiers (the range of the asserted table $T : R \rightarrow 2^C$), and F the allowed framework set. A scan produces a typed graph $G = (N, E)$ with provenance $p(\cdot)$ on every node, edge, and mapping, and a set of action maps $M = M_{\text{asserted}} \cup M_{\text{inferred}} \cup M_{\text{ambiguous}} \cup M_{\text{none}}$. Validation is a predicate $V(G, M)$; scoring is $S(M, G) \rightarrow [0, 100] \times \text{Tier}$; $H = \text{sha256}(\text{canon}(\text{evidence}))$ and Sig is the hybrid signature over $\text{canon}(\text{body})$ where body commits to H .

The ten invariants the implementation maintains (the v1 set, plus the two forgery backstops shipped in v0.2):

- I1. $m \in M_{\text{asserted}} \Rightarrow m.\text{reason} \in R$ declared in source (no asserted without a declared reason)
- I2. $m \in M_{\text{asserted}} \Rightarrow m.\text{id} \in C$ (asserted id must exist in the fixed table)
- I3. heuristic match $\Rightarrow$ confidence in {inferred, ambiguous} (a heuristic can never be asserted)
- I4. graph match $\Rightarrow$ confidence in {GRAPH_INFERRED_*} (a graph candidate can never be asserted)
- I5. advisory mappings do not affect coverage, rigor, or breadth
- I6. evidence changes $\Rightarrow$ $\text{verify}(\text{receipt}) = \text{false}$ (tamper-evidence)
- I7. not $V(G, M) \Rightarrow$ no badge is emitted (fail-closed)
- I8. remediation after-state is labeled PROJECTED unless wired and re-scanned
- I9. $m.\text{fw} \text{ not in } F \text{ or } (m.\text{fw}, m.\text{id}) \text{ not in } T \Rightarrow V = \text{false}$ (E_FRAMEWORK / E_FAKE_ID)
- I10. $\text{matched_via} = \text{asserted_table with empty mappings} \Rightarrow V = \text{false}$ (E_MATCHED_VIA: no claim without evidence)

I1-I4 are the honesty core: the only path to ASSERTED is a source declaration, and every automatic layer (verb heuristic in Section 3.2, graph resolver in Section 9) is structurally barred from it. I9-I10 are the forgery backstops: even a hand-edited ontology that smuggles a fake id or a bare upgraded `matched_via` is rejected at validation, before it can reach the badge or the receipt. The set of asserted identifiers a run can emit is exactly `range(T)`, independent of input, which is why the no-fabrication property is structural and not a test artifact.

5. Claims and non-claims

We separate what OAO measures from what it does not, so the headline cannot be over-read.

Claims.

- **C1.** OAO extracts a static action surface from supported source formats without executing the target.
- **C2.** OAO emits a deterministic typed graph and a signed, offline-verifiable receipt (Sections 7.1, 7.4).
- **C3.** OAO's validation prevents fabricated asserted control identifiers and forged upgrades (I2, I9, I10).
- **C4.** OAO distinguishes declared (ASSERTED) mappings from heuristic guesses (INFERRED/AMBIGUOUS) on every action (I1-I3).
- **C5.** OAO identifies the absence of machine-readable governance declarations in scanned targets, and which actions remain ungoverned even under inference (Section 7.2).
- **C6.** A graph-grounded resolver can supply confidence-tagged candidate controls and live adversary signals, which remain non-asserted unless source-declared or human-confirmed (Section 9).

- **C7.** OAO composes with a runtime policy engine: an action behind a gate with a canonical deny reason scores ASSERTED, and OAO's findings export as SARIF for existing pipelines.

Non-claims.

- **N1.** OAO does not prove runtime enforcement. It reads source; a control present in source may be bypassed at runtime, and a control applied only at runtime is invisible to it.
- **N2.** OAO does not prove legal compliance with any framework.
- **N3.** OAO does not prove an agent is safe.
- **N4.** OAO does not see actions behind unsupported dynamic tool registration.
- **N5.** OAO's ten canonical reasons are one vendor's vocabulary and are not claimed complete.
- **N6.** The two-agent real-world result is not a population estimate.
- **N7.** Projected remediation (Section 7.5) is not runtime enforcement.

6. Data and experimental setup

We scanned two classes of agent and ran a battery of measured controls. All numbers in Section 7 are produced by `paper/experiments.py` (figures and `results.json`) or transcribed from committed signed scans; nothing is hand-entered.

Reference and fixture agents (in-repo, known ground truth). Three example agents (`sample_agent`, `hardened_agent`, `agent_fde` - the remediation agent itself, defined to the standard) and five conformance fixtures authored to a known governance state (`shell_exec_agent`, `telemetry_agent`, `payment_agent` with a declared `dual_control_required` reason, `ambiguous_run_agent`, and `fake_control_agent` which claims a fabricated NIST AC-999). Because we authored these to a known state, they form a small labeled set against which extraction and classification accuracy can be measured honestly.

Real-world agents (committed signed scans). The two most-starred autonomous coding agents on GitHub, scanned at pinned upstream commits, with full output committed under `docs/scans/`: `OpenInterpreter/open-interpreter` at `e00f08e` (~58k stars) and `gpt-engineer-org/gpt-engineer` at `a90fcd5` (~54k stars). Star counts are approximate, move over time, and are not part of the signed evidence. For this revision we made one fresh blobless clone of open-interpreter at the pinned commit, only to (a) reproduce the committed scan (Section 7.4) and (b) run the remediation agent (Section 7.5); the committed scans were not modified.

Non-agentic baseline. A 12-file corpus of pure-compute Python (Fibonacci, quicksort, matrix multiply, dynamic-programming LCS, bisection search, Roman-numeral formatting, etc.) with no I/O, network, or side effects, used to measure the false-positive rate of the action-surface extractor (Section 7.6).

Reproducibility and honesty controls. The pipeline is deterministic - the same OAO version on the same input yields the same ontology and the same `evidence_hash`, which we measure directly (Section 7.1). Analysis is static: AST parse plus text read, no execution, no network call in the core pipeline. The test suite - **152 tests, all passing at paper time** (measured; v0.2 grew the suite from the 93 reported in v1 of this paper and the 80 in an earlier README) - mechanically asserts the honesty guarantees, including golden fixtures, mutation tests for forged upgrades, and an independent JavaScript canonicalizer that reproduces the receipt hash so a browser verifier agrees byte-for-byte.

What we did not run, and why. We report this plainly. We do *not* report extraction precision/recall on the two real-world agents, because doing so honestly requires a security engineer to label which of their functions are truly

side-effecting actions, and we have no such ground-truth labels; the measured precision/recall we report (Section 7.1) is on the in-repo set we authored. We do not report calibration error for the graph-grounded resolver, multi-annotator agreement, or a large-corpus prevalence estimate. These are the open benchmark program (Sections 8, 9), and inventing numbers for them would violate our own honesty invariants.

7. Results

All figures are generated from measured data by `paper/experiments.py`. Reference/fixture rows are in-process runs at paper time; real-world rows are transcribed from the committed signed scans.

7.1 Determinism and classification accuracy (measured)

Three measured controls establish that the instrument is stable and, on data where we have ground truth, accurate:

- **Determinism: 8/8.** Each in-repo agent was scanned twice; the `evidence_hash` reproduced byte-for-byte every time (reproducibility rate 1.0). This is the property the receipt depends on.
- **Tier-prediction accuracy: 7/7.** On the seven in-repo agents with an unambiguous known tier, the tool predicted the correct tier every time (the eighth, `payment_agent`, is governed but its exact band is not pinned, so it is excluded from this count rather than asserted).
- **Action governance-state accuracy: 10/10.** On a hand-labeled set of ten actions spanning all four outcomes (a declared `dual_control_required` -> ASSERTED, `send_telemetry` / `export_records` / `delete_user` / `deploy_release` -> INFERRED, `run_pipeline` / `process_item` -> AMBIGUOUS-or-UNGOVERNED, `exec` / `post_data` -> UNGOVERNED), the tool classified every one correctly.

These are small-N by construction - they are exactly the cases where we legitimately know the answer - and we do not extrapolate them to the real-world agents.

7.2 The headline table and the governance-state breakdown

Agent	Stars*	Tier	Score	Governed	Declared	Inferred	Ambiguous	Ungoverned	Assert FWs
<code>sample_agent</code>	-	UNGOVERNED	41	16	3	10	0	3	3
<code>hardened_agent</code>	-	SOVEREIGN	93	15	14	0	0	1	3
<code>agent_fde</code>	-	SOVEREIGN	94	18	17	0	0	1	3
<code>open- interpreter @e00f08e</code>	~58k	UNGOVERNED	15	21	0	11	9	1	0
<code>gpt-engineer @a90fcd5</code>	~54k	UNGOVERNED	15	6	0	3	2	1	0

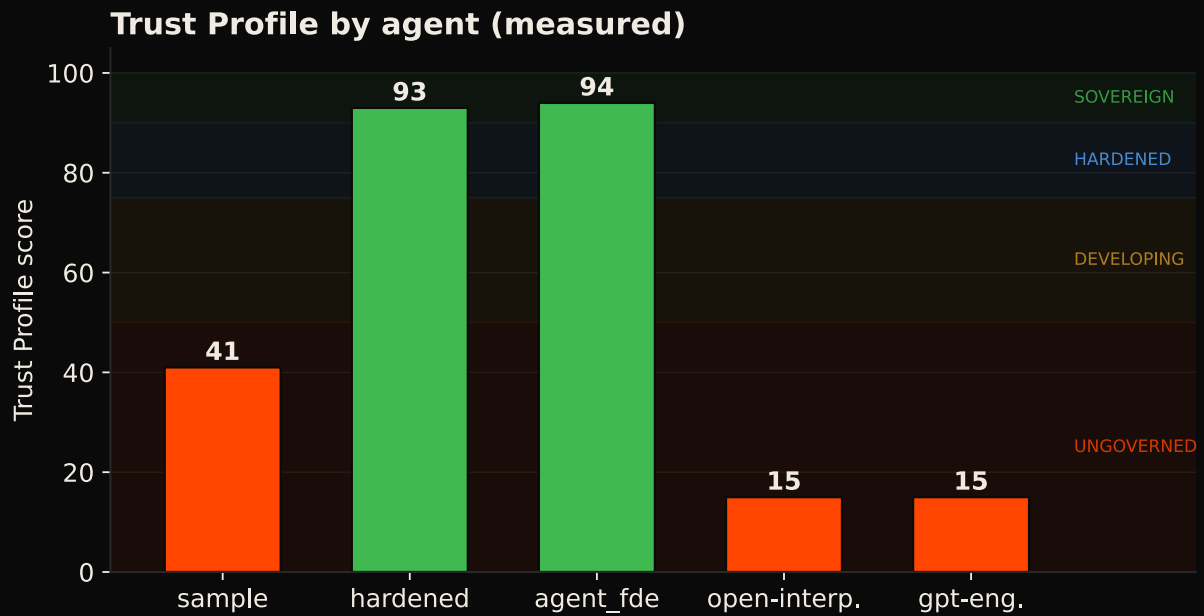


Figure 1. Trust Profile score by agent, with tier-band shading. The two real-world agents floor in the UNGOVERNED band; the two declared reference agents reach SOVEREIGN.

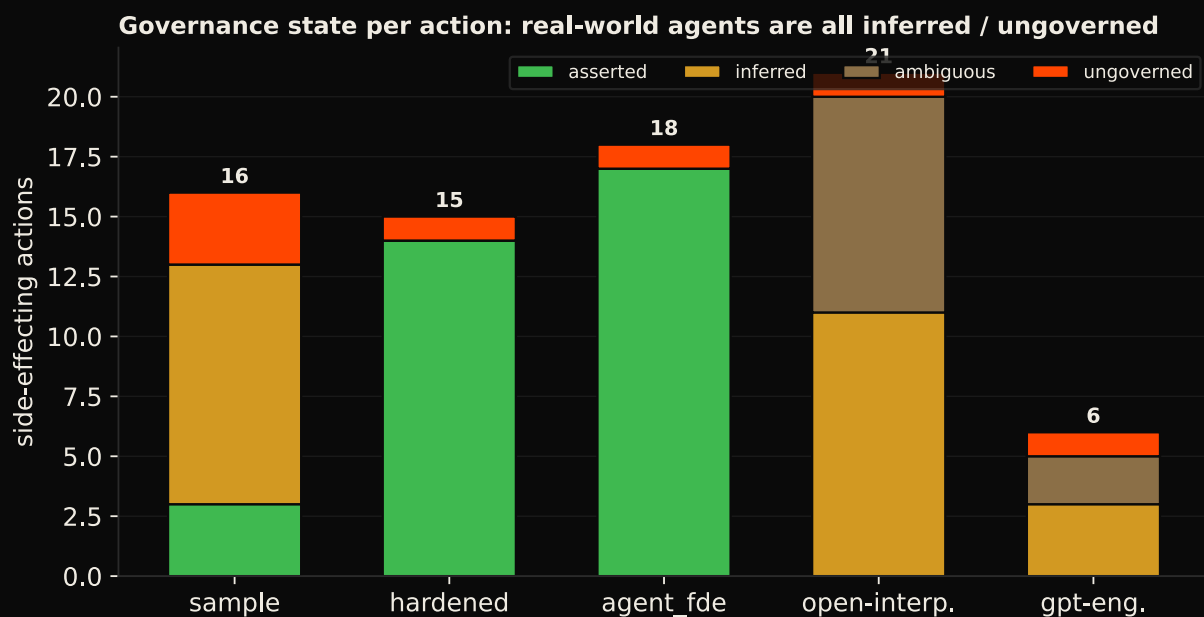


Figure 3. Governance state of every side-effecting action. The two real-world agents carry zero ASSERTED actions; their surface is entirely INFERRED, AMBIGUOUS, or UNGOVERNED. This is the central finding rendered per action.

Figure 3 is the central result: for open-interpreter, of 21 side-effecting actions, 0 are ASSERTED, 11 INFERRED, 9 AMBIGUOUS, and 1 (`exec`) UNGOVERNED; for gpt-engineer, 0 of 6 are ASSERTED. Figure 2 shows why the score floors - coverage and rigor are zero for both real-world agents because every mapping is heuristic.

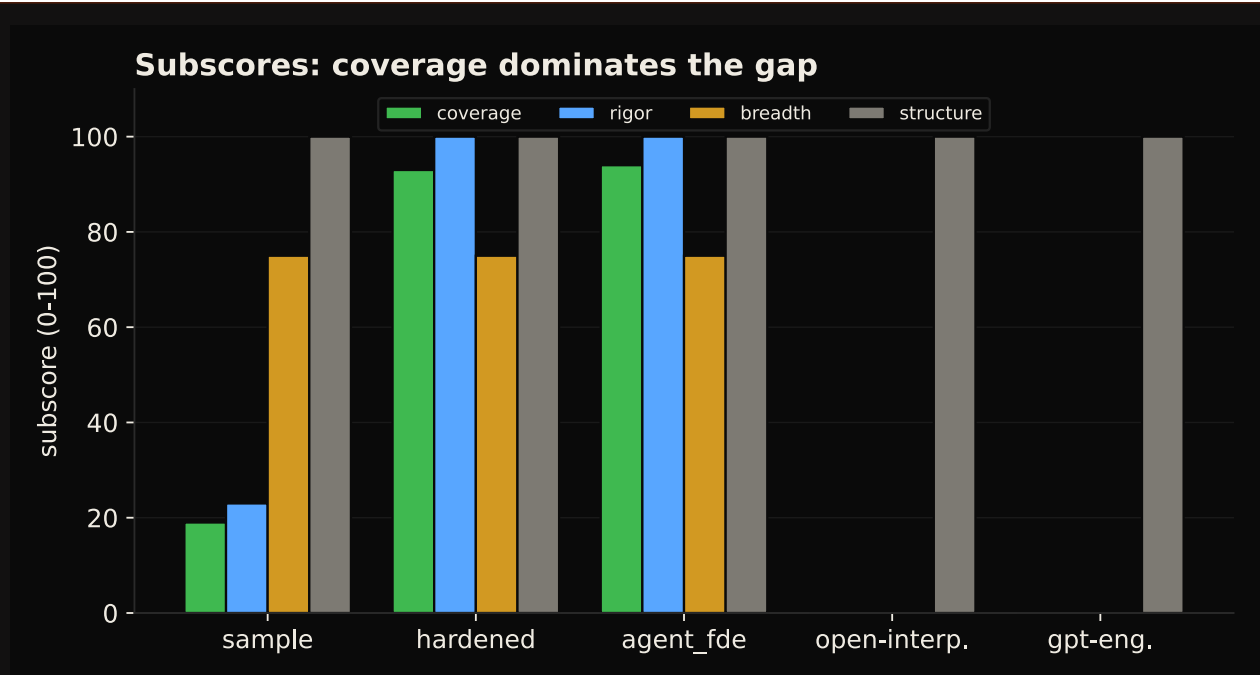


Figure 2. Subscores per agent. Coverage (the dominant weight) is zero for both real-world agents; structure is 100 because the extracted graph is fully typed and connected - a well-formed map of an ungoverned surface.

7.2.1 The headline finding

The two most-starred autonomous coding agents on GitHub carry zero declared governance bindings, and the single most dangerous action in each is ungoverned even by inference. We state it this way, and not as "zero asserted controls," because a zero-asserted outcome is structurally guaranteed for any agent that does not declare reasons (Section 3.2); neither agent declares any. The measured, *non-structural* findings are the anchors: in open-interpreter, `exec` (arbitrary code execution) matched no canonical reason and no heuristic verb - MITRE ATT&CK catalogs this behavior as T1059, the technique a SOC already hunts, yet nothing in the agent's own source answers for it. In gpt-engineer, the parallel action is `post_data`. And the inferred-only surface (egress actions landing on AC-4/SC-7 and T1048 only as guesses) is, for every one of those actions, now a manual confirmation task a declaration would have removed.

7.3 Tier assignments are robust to the dominant weight (measured)

We re-scored all five agents from their measured subscores under a sweep of the coverage weight over $\{0.35, 0.40, 0.45, 0.50, 0.55\}$, renormalizing the remaining mass across rigor/breadth/structure in their original 0.25/0.15/0.15 ratio. The measured result: **every agent keeps its tier at every grid point, and the score ordering is preserved.** Integer scores move modestly - `sample_agent` spans 44 down to 37, the two real-world agents 18 down to 12 - but no agent crosses a band. The one boundary effect: at coverage weight 0.35, `agent_fde` and `hardened_agent` tie at 93 (both SOVEREIGN); elsewhere `agent_fde` leads 94 to 93. This is the measured basis for treating tiers as the primary result.

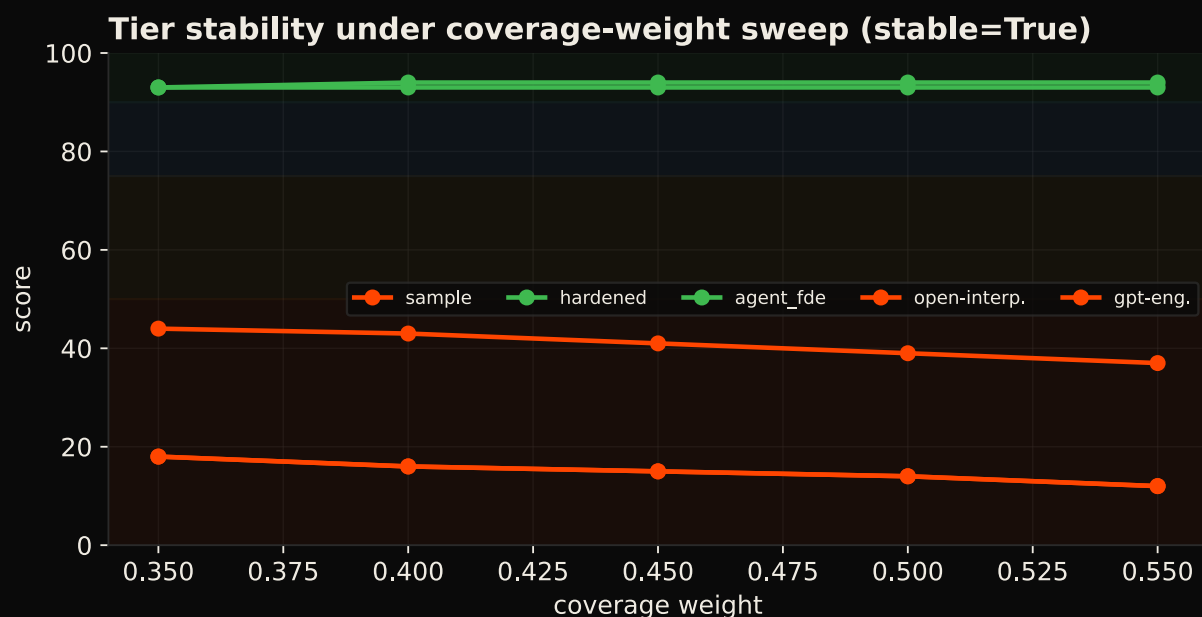


Figure 4. Weight-sensitivity sweep. Each line is one agent's score as the coverage weight varies; tier bands are shaded. No line crosses a band boundary, so tier assignments do not depend on the exact weight within this range.

7.4 The receipt is verifiable, tamper-evident, and independently reproduced

We verified both committed real-world receipts from the certificate alone, with no network, then forged a one-field upgrade and re-verified. The measured tamper matrix:

Receipt	Clean verify	Ed25519 leg	One-field tamper (<code>matched_via</code> <code>none</code> -> <code>asserted_table</code>)
<code>open-interpreter</code>	<code>ok: True</code>	verified	<code>ok: False</code> - "evidence_hash mismatch -- evidence was altered"
<code>gpt-engineer</code>	<code>ok: True</code>	verified	<code>ok: False</code> - "evidence_hash mismatch -- evidence was altered"

Both committed receipts additionally now carry all three signature legs (Ed25519 + ML-DSA-65 + SLH-DSA), upgraded purely additively so the original `evidence_hash`, `atom_id`, and Ed25519 signature are byte-identical to what v1 reported; a release-gate script verifies all three legs on both. We also reproduced determinism end-to-end: a fresh blobless clone of open-interpreter at `e00f08e2ed54e97a530326cc2b1a93389bbe5f7c`, scanned by its folder name, reproduced the committed `evidence_hash` byte-for-byte (`433522f130...`), the identical `atom_id` (`oao-OPENINTERPRE-433522f130`), and the identical verdict; only the signature differs, because the re-scan signs with a local key.

7.5 Remediation as a measured before/after, including a real-world agent

AgentFDE runs the full loop autonomously - scan, triage, generate governance (an agent declaration plus a fail-closed Rego stub), re-score, notarize - without ever executing the target. Every "after" here is the tool's measured re-score *with the generated declarations bound*; the receipt's decision string is `FDE_REMEDIATION_PROJECTED`, and the state is PROJECTED until the governance is wired into the agent and re-scanned (I8).

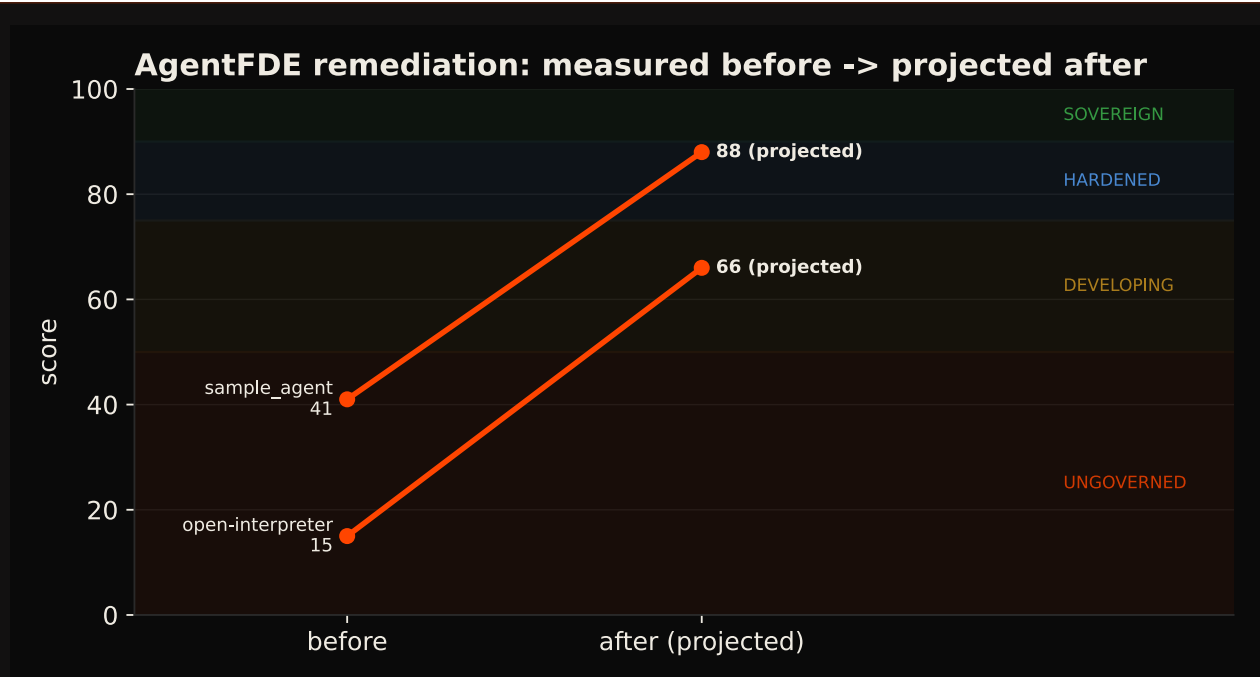


Figure 5. Remediation before/after. AgentFDE lifts the reference agent from UNGOVERNED 41 to a projected HARDENED 88, and the real open-interpreter clone from UNGOVERNED 15 to a projected DEVELOPING 66 - one tier band, with the ambiguous nine left to a human.

On `sample_agent`: 16 actions, 3 asserted before; AgentFDE produced 10 auto-remediations and left 3 raw-verb actions for a human (UNGOVERNED 41 -> projected HARDENED 88). On the real open-interpreter clone (before-state independently reproducing the committed verdict): 12 remediations, 11 automatic (the egress family to `regulated_egress_blocked`, the three `migrate_*` to `approval_required`, `delete_event` to `change_freeze_active`, `terminate` to `human_review_required`) and 1 proposed requiring human confirmation - `exec` to `approval_required`, which AgentFDE will not auto-assert. Nine actions remain open: the entire `run_*` family, whose overloaded verb the crosswalk deliberately refuses to claim. UNGOVERNED 15 -> projected DEVELOPING 66. We report this exactly as measured: autonomous remediation lifts the tier one band and honestly hands the ambiguous nine to a human. AgentFDE is itself a governed agent that scores SOVEREIGN 94 under the same rubric.

7.6 False-positive baseline: pure-compute code raises no action surface (measured)

A frequent objection to any "find the dangerous actions" tool is that it cries wolf. We measured the opposite bound: scanning the 12-file non-agentic corpus (pure algorithms, no I/O), the extractor produced **zero** side-effecting actions - a false-positive rate of 0.0 on this baseline. The extractor keys on side-effect verbs and tool decorators, so ordinary computation does not register as an agentic action surface. This is the lower bound that makes the UNGOVERNED findings legible: when the tool reports a side-effecting surface, it is because side-effecting actions are present.

Baseline: pure-compute code yields no agentic action surface



Figure 6. Non-agentic baseline: 12 pure-compute files scanned, 0 spurious side-effecting actions (false-positive rate 0.0). The instrument does not manufacture an action surface where there is none.

7.7 Interpretation

A low tier does not mean the tool failed; it means the agent's actions carry no declared governance bindings, which is the honest finding for nearly every agent shipping today. The contrast across the table makes the mechanism legible: the same archetype lands at **UNGOVERNED** with partial declarations (`sample_agent`) and **SOVEREIGN** once every action declares a reason (`hardened_agent`), while the real-world agents - declaring nothing - sit at the bottom of the band. The tier moves only when declared, sourced governance is present (Section 7.5), it is robust to the dominant weight (Section 7.3), it is deterministic and accurate where we have ground truth (Section 7.1), and it does not fire on non-agentic code (Section 7.6).

8. Threats to validity

We state these plainly; an instrument that maps governance has to be honest about its own coverage.

Static analysis cannot see runtime behavior (N1). OAO reads source, not execution. An action gated only by runtime configuration or an external policy server OAO does not ingest will look ungoverned even if controlled in deployment; conversely, a source control may be bypassed at runtime. OAO is the pre-flight inspection, not the flight recorder, and composes with a runtime gate rather than replacing it.

Runtime tool registration under-extracts (N4). OAO extracts decorators and verb-named functions; it does not yet resolve tools built inside a `list_tools()` handler at runtime. Scanning such a target under-extracts. This is documented, not silent, and is on the roadmap.

The INFERRED heuristic is verb-regex brittle. A dangerous action with an innocuous name may fall to the weak/ambiguous path or to **UNGOVERNED**, and a benign action with a strong verb may be downgraded onto controls that do not apply. The design contains the cost: brittleness reduces recall/precision in the INFERRED layer but cannot fabricate an asserted control (I3). The honest claim is narrow - asserted coverage is trustworthy; inferred coverage is a prompt to confirm. The graph-grounded resolver (Section 9) is the principled answer to this, and its candidates are likewise non-asserting (I4).

The crosswalk is single-source (N5). The asserted table is one fixed mapping from ten reasons to controls. Two GRC teams may map a reason differently, and published control text changes. Every output ships the note that mappings are proposed and must be confirmed; enterprise control ids are mapped at onboarding, never auto-asserted.

Selection and labeling limits (N6). The headline real-world result rests on two agents chosen for being most-starred; this is a finding about these two at these commits, not a prevalence. Critically, **we did not measure extraction precision/recall on these two agents**, because we have no ground-truth labels for which of their functions are truly side-effecting; the precision/recall we do report (7/7 tier, 10/10 action-state, Section 7.1) is on the in-repo set we authored, and we do not extrapolate it. Producing labeled real-world ground truth is the first item of the benchmark program below.

What the present evaluation does not yet include. To be explicit about coverage: this paper reports determinism, classification accuracy on a labeled in-repo set, weight-sensitivity, a tamper matrix, a remediation before/after, and a non-agentic false-positive baseline. It does *not* yet include (a) a human-labeled real-world corpus with extraction precision/recall and inter-annotator agreement; (b) calibration error (ECE) for the graph-grounded resolver's confidences; (c) an adversarial fixture suite (misleading names, encoded commands, indirect egress) reported as a detection rate; or (d) a sampled multi-agent corpus converting the two-agent finding into a prevalence with a stated confidence. These are designed but not run, and are the subject of Section 9.

Scoring weights are a design choice. The 0.45/0.25/0.15/0.15 weights and tier floors encode a judgment that coverage dominates. The measured sweep (Section 7.3) shows tier assignments and ordering are unchanged for coverage weights from 0.35 to 0.55, but it perturbs one weight over one range on five agents and does not vary the floors.

9. Discussion and future work

The measured results suggest the governance gap is not an edge case but the modal state of deployed autonomous agents. Making that gap a typed, signed tier changes what a buyer, an auditor, or a CI gate can do with it; the same machinery already runs as a pull-request check that fails when the tier drops or a new EU AI Act Article 14 gap appears.

The graph-grounded semantic resolver (now shipped, not future). v1 of this paper listed a graph-grounded crosswalk as future work; it now exists as a hosted resolver. For each action it builds a normalized shape, anchors a MITRE technique deterministically, expands curated kernel edges (technique -> tactic, and skill -> work-role categories that name who should own the gate), retrieves live adversary signals (current CVEs of the same risk class - retrieved, never generated), and proposes canonical reasons and controls. Crucially, every candidate is emitted at a **GRAPH_INFERRED** confidence that the invariants bar from ever becoming ASSERTED (I4): only a source declaration or a confirmed human governance record asserts. This directly addresses the verb-regex brittleness threat - the nine `run_*` actions AgentFDE left open on open-interpreter are exactly the overloaded-verb cases a semantic layer can triage where a regex honestly cannot - while preserving the asserted invariant. Its own evidence is signed with the same hybrid post-quantum receipt and embeds a graph-snapshot fingerprint and resolver version so a resolution is reproducible evidence, not an unfalsifiable claim. Calibrating its confidences against a labeled set (reporting ECE and top-k recall) is the next measured step.

A verified-or-abstain lattice boundary (now wired). The graph-grounded candidates feed a deterministic decision boundary that gives the runtime gate its escalate signal. We represent the epistemic state of an action as a powerset lattice over the ten canonical reasons (abstract interpretation in the sense of Cousot & Cousot 1977): top is

every reason still open, bottom is no consistent reason, and a fixed threshold-elimination operator descends toward a precise state. The lattice region maps directly onto the taxonomy and onto a gate action -- a source-declared narrow state is VERIFIED (PROCEED, the policy engine decides allow/deny); an inferred-only narrow state is PROCEED-FLAGGED, or ESCALATE on a critical risk domain; a wide state (competing interpretations) or the bottom state (no control) is ABSTAIN -> ESCALATE to a human. This formalizes the INFERRED-vs-AMBIGUOUS boundary as lattice width against the threshold, and the escalate trigger as the bottom-state conflict detector, rather than a bare heuristic. It is deliberately DETERMINISTIC, not a learned model: that keeps it auditable and lets it compose UNDER the signed receipt -- it records the abstain decision into the receipt, it never replaces the cryptographic proof. On the committed open-interpreter scan, the boundary escalates 15 of 21 actions, the honest consequence of an agent that declares no governance: almost nothing is grounded enough to act on autonomously.

The benchmark program (the honest gap this paper leaves open). Converting the two-agent finding into a measured prevalence requires (1) a human-labeled real-world corpus (a security engineer marking the true side-effecting surface of each agent) to report extraction precision/recall with inter-annotator agreement; (2) an adversarial fixture tier (misleading names, encoded commands, indirect egress, fake policies) reported as a detection rate; and (3) calibration metrics for the graph layer. We name these rather than approximate them.

The hosted notary, registry, and the runtime gate it composes with. The open-source tool mints a self-signed receipt offline; a hosted notary and registry add cross-organization verification and a public trust ledger so a receipt minted by one team is checkable by another. And because OAO is the static map and not the lock, the natural complement is a runtime policy engine: exposing the scan as an MCP tool lets an agent platform ask, in-loop, whether a proposed action has a declared control before it executes, routing the answer to the gate where enforcement belongs. OAO finds the doors with no lock; the runtime gate is the lock. The open-core boundary - local deterministic tool, hosted graph resolver and registry on top - is explicit in the standard.

10. Conclusion

Autonomous agents act faster than review and, for most deployments, with no machine-readable record of which control answers for each action. OpenAgentOntology supplies that record: it reads an agent's own source without executing it, types the result as a graph, maps each side-effecting action to NIST 800-53, the EU AI Act, the OWASP LLM Top 10, the NIST AI RMF, and MITRE ATT&CK through a three-layer crosswalk that can never fabricate a control id, scores a four-tier Trust Profile, and signs the whole thing into a hybrid post-quantum receipt that verifies offline and fails on a single altered byte. Pointed at the two most-starred autonomous coding agents on GitHub, it found zero declared governance bindings across their 21 and 6 side-effecting actions - every control inferred from a verb name, and the most dangerous action in each ungoverned even by inference. We confirmed the result with measured controls: deterministic re-scans, correct tier and action-state classification where ground truth exists, a tamper matrix in which both receipts fail under a one-field edit, a coverage-weight sweep that moves no agent across a tier band, and a non-agentic baseline that raises no spurious action surface. We showed the gap is closeable in the same currency - an autonomous remediation agent lifted a real open-interpreter clone one tier band, honestly leaving nine ambiguous actions to a human - and a deterministic lattice boundary turns each resolution into a verified-or-abstain gate verdict, escalating 15 of the agent's 21 actions to a human because almost nothing it does is grounded enough to act on autonomously. We were explicit about the benchmark program we have not yet run. The Agent Ontology is the accountability layer the agent stack skipped; this work makes it measurable, makes the measurement something you cannot fake, and keeps it honest about its own edges. It is the map that finds the doors with no lock - and it composes with the lock rather than pretending to be it.

Logs explain. Receipts prove.

References (standards by identifier): NIST SP 800-53r5 · NIST AI RMF 1.0 · Regulation (EU) 2024/1689 (EU AI Act) · OWASP LLM Top 10 (2025) · MITRE ATT&CK · NIST FIPS 204 (ML-DSA) · NIST FIPS 205 (SLH-DSA) · Ed25519 (RFC 8032) · OASIS SARIF 2.1.0 · NIST OSCAL · Sigstore / SLSA · OpenTelemetry · CycloneDX / SPDX. All control identifiers are cited by their own published names; OAO never constructs a framework id and emits only identifiers present in its fixed asserted table. Mappings are proposed and confidence-tagged; confirm against current published control text before relying on them for audit.

Reproduce every figure and number in this paper with `PYTHONPATH=. python paper/experiments.py` from the repository root.